3
Fundamentals of Software Engineering for Games

In this chapter, we'll discuss the foundational knowledge needed by any professional game programmer. We'll explore numeric bases and representations, the components and architecture of a typical computer and its CPU, machine and assembly language, and the C++ programming language. We'll review some key concepts in object-oriented programming (OOP), and then delve into some advanced topics that should prove invaluable in any software engineering endeavor (and especially when creating games). As with Chapter 2, some of this material may already be familiar to some readers. However, I highly recommend that all readers at least skim this chapter, so that we all embark on our journey with the same set of tools and supplies.

3.1 C++ Review and Best Practices

Because C++ is arguably the most commonly used language in the game industry, we will focus primarily on C++ in this book. However, most of the concepts we'll cover apply equally well to *any* object-oriented programming language. Certainly a great many other languages are used in the game industry—imperative languages like C; object-oriented languages like C# and

Java; scripting languages like Python, Lua and Perl; functional languages like Lisp, Scheme and F#, and the list goes on. I highly recommend that every programmer learn at least two high-level languages (the more the merrier), as well as learning at least some *assembly language* programming (see Section 3.4.7.3). Every new language that you learn further expands your horizons and allows you to think in a more profound and proficient way about programming overall. That being said, let's turn our attention now to object-oriented programming concepts in general, and C++ in particular.

3.1.1 Brief Review of Object-Oriented Programming

Much of what we'll discuss in this book assumes you have a solid understanding of the principles of object-oriented design. If you're a bit rusty, the following section should serve as a pleasant and quick review. If you have no idea what I'm talking about in this section, I recommend you pick up a book or two on object-oriented programming (e.g., [7]) and C++ in particular (e.g., [46] and [36]) before continuing.

3.1.1.1 Classes and Objects

A *class* is a collection of attributes (data) and behaviors (code) that together form a useful, meaningful whole. A class is a *specification* describing how individual *instances* of the class, known as *objects*, should be constructed. For example, your pet Rover is an instance of the class "dog." Thus, there is a one-to-many relationship between a class and its instances.

3.1.1.2 Encapsulation

Encapsulation means that an object presents only a limited interface to the outside world; the object's internal state and implementation details are kept hidden. Encapsulation simplifies life for the user of the class, because he or she need only understand the class' limited interface, not the potentially intricate details of its implementation. It also allows the programmer who wrote the class to ensure that its instances are always in a logically consistent state.

3.1.1.3 Inheritance

Inheritance allows new classes to be defined as *extensions* to preexisting classes. The new class modifies or extends the data, interface and/or behavior of the existing class. If class `Child` extends class `Parent`, we say that `Child` *inherits from* or is *derived from* `Parent`. In this relationship, the class `Parent` is known as the *base class* or *superclass*, and the class `Child` is the *derived class*

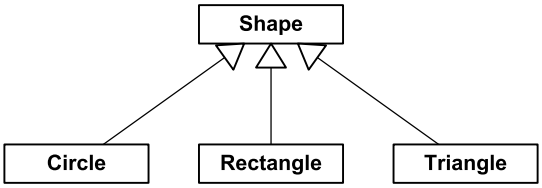

Figure 3.1. UML static class diagram depicting a simple class hierarchy.

or *subclass*. Clearly, inheritance leads to hierarchical (tree-structured) relationships between classes.

Inheritance creates an "is-a" relationship between classes. For example, a circle *is a* type of shape. So, if we were writing a 2D drawing application, it would probably make sense to derive our `Circle` class from a base class called `Shape`.

We can draw diagrams of class hierarchies using the conventions defined by the Unified Modeling Language (UML). In this notation, a rectangle represents a class, and an arrow with a hollow triangular head represents inheritance. The inheritance arrow points from child class to parent. See Figure 3.1 for an example of a simple class hierarchy represented as a UML *static class diagram*.

Multiple Inheritance

Some languages support *multiple inheritance* (MI), meaning that a class can have more than one parent class. In theory MI can be quite elegant, but in practice this kind of design usually gives rise to a lot of confusion and technical difficulties (see http://en.wikipedia.org/wiki/Multiple_inheritance). This is because multiple inheritance transforms a simple *tree* of classes into a potentially complex *graph*. A class graph can have all sorts of problems that never plague a simple tree—for example, the *deadly diamond* (http://en.wikipedia.org/wiki/Diamond_problem), in which a derived class ends up containing *two copies* of a grandparent base class (see Figure 3.2). (In C++, *virtual inheritance* allows one to avoid this doubling of the grandparent's data.) Multiple inheritance also complicates casting, because the actual address of a pointer may change depending on which base class it is cast to. This happens because of the presence of multiple vtable pointers within the object.

Most C++ software developers avoid multiple inheritance completely or only permit it in a limited form. A common rule of thumb is to allow only simple, parentless classes to be multiply inherited into an otherwise strictly single-inheritance hierarchy. Such classes are sometimes called *mix-in classes*

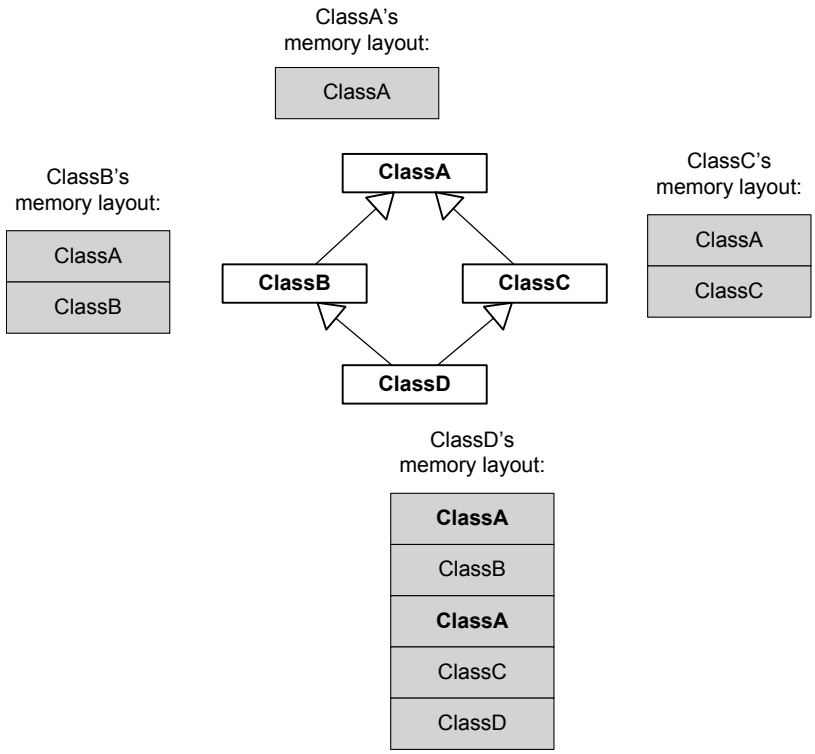

Figure 3.2. "Deadly diamond" in a multiple inheritance hierarchy.

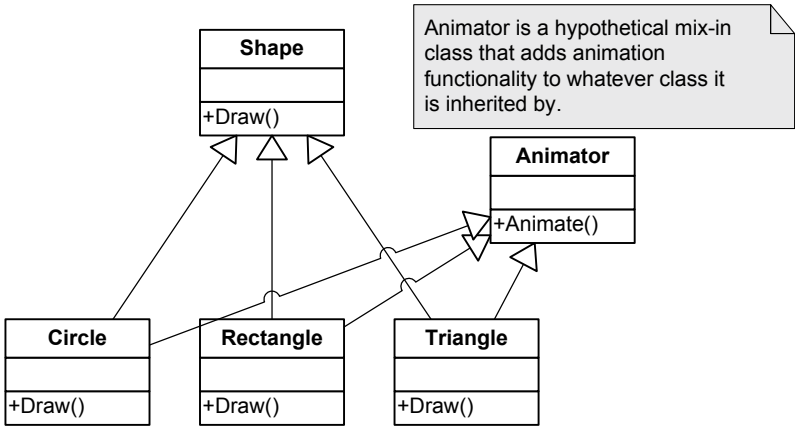

Figure 3.3. Example of a mix-in class.

because they can be used to introduce new functionality at arbitrary points in a class tree. See Figure 3.3 for a somewhat contrived example of a mix-in class.

3.1.1.4 Polymorphism

Polymorphism is a language feature that allows a collection of objects of different types to be manipulated through a single *common interface*. The common interface makes a heterogeneous collection of objects *appear* to be homogeneous, from the point of view of the code using the interface.

For example, a 2D painting program might be given a list of various shapes to draw on-screen. One way to draw this heterogeneous collection of shapes is to use a switch statement to perform different drawing commands for each distinct type of shape.

```
void drawShapes(std::list<Shape*>& shapes)
{
   std::list<Shape*>::iterator pShape = shapes.begin();
   std::list<Shape*>::iterator pEnd = shapes.end();

   for ( ; pShape != pEnd; pShape++)
   {
      switch (pShape->mType)
      {
      case CIRCLE:
         // draw shape as a circle
         break;

      case RECTANGLE:
         // draw shape as a rectangle
         break;

      case TRIANGLE:
         // draw shape as a triangle
         break;

      //...
      }
   }
}
```

The problem with this approach is that the `drawShapes()` function needs to "know" about all of the kinds of shapes that can be drawn. This is fine in a simple example, but as our code grows in size and complexity, it can become difficult to add new types of shapes to the system. Whenever a new shape

type is added, one must find every place in the code base where knowledge of the set of shape types is embedded—like this switch statement—and add a case to handle the new type.

The solution is to insulate the majority of our code from any knowledge of the types of objects with which it might be dealing. To accomplish this, we can define classes for each of the types of shapes we wish to support. All of these classes would inherit from the common base class `Shape`. A *virtual function*—the C++ language's primary polymorphism mechanism—would be defined called `Draw()`, and each distinct shape class would implement this function in a different way. Without "knowing" what specific types of shapes it has been given, the drawing function can now simply call each shape's `Draw()` function in turn.

```
struct Shape
{
   virtual void Draw() = 0; // pure virtual function
   virtual ~Shape() { } // ensure derived dtors are virtual
};

struct Circle : public Shape
{
   virtual void Draw()
   {
      // draw shape as a circle
   }
};

struct Rectangle : public Shape
{
   virtual void Draw()
   {
      // draw shape as a rectangle
   }
};

struct Triangle : public Shape
{
   virtual void Draw()
   {
      // draw shape as a triangle
   }
};
```

```
void drawShapes(std::list<Shape*>& shapes)
{
   std::list<Shape*>::iterator pShape = shapes.begin();
   std::list<Shape*>::iterator pEnd = shapes.end();

   for ( ; pShape != pEnd; pShape++)
   {
      pShape->Draw(); // call virtual function
   }
}
```

3.1.1.5 Composition and Aggregation

Composition is the practice of using a *group* of *interacting* objects to accomplish a high-level task. Composition creates a "has-a" or "uses-a" relationship between classes. (Technically speaking, the "has-a" relationship is called *composition*, while the "uses-a" relationship is called *aggregation*.) For example, a spaceship *has an* engine, which in turn *has a* fuel tank. Composition/aggregation usually results in the individual classes being simpler and more focused. Inexperienced object-oriented programmers often rely too heavily on inheritance and tend to underutilize aggregation and composition.

As an example, imagine that we are designing a graphical user interface for our game's front end. We have a class `Window` that represents any rectangular GUI element. We also have a class called `Rectangle` that encapsulates the mathematical concept of a rectangle. A naïve programmer might derive the `Window` class from the `Rectangle` class (using an "is-a" relationship). But in a more flexible and well-encapsulated design, the `Window` class would *refer to* or *contain* a `Rectangle` (employing a "has-a" or "uses-a" relationship). This makes both classes simpler and more focused and allows the classes to be more easily tested, debugged and reused.

3.1.1.6 Design Patterns

When the same type of problem arises over and over, and many different programmers employ a very similar solution to that problem, we say that a *design pattern* has arisen. In object-oriented programming, a number of common design patterns have been identified and described by various authors. The most well-known book on this topic is probably the "Gang of Four" book [19].

Here are a few examples of common general-purpose design patterns.

- *Singleton*. This pattern ensures that a particular class has only one instance (the *singleton instance*) and provides a global point of access to it.
- *Iterator*. An iterator provides an efficient means of accessing the individual elements of a collection, without exposing the collection's underlying

implementation. The iterator "knows" the implementation details of the collection so that its users don't have to.

- *Abstract factory*. An abstract factory provides an interface for creating families of related or dependent classes without specifying their concrete classes.

The game industry has its own set of design patterns for addressing problems in every realm from rendering to collision to animation to audio. In a sense, this book is all about the high-level design patterns prevalent in modern 3D game engine design.

Janitors and RAII

As one very useful example of a design pattern, let's have a brief look at the "resource acquisition is initialization" pattern (RAII). In this pattern, the acquisition and release of a resource (such as a file, a block of dynamically allocated memory, or a mutex lock) are bound to the constructor and destructor of a class, respectively. This prevents programmers from accidentally forgetting to release the resource—you simply construct a local instance of the class to acquire the resource, and let it fall out of scope to release it automatically. At Naughty Dog, we call such classes *janitors* because they "clean up" after you.

For example, whenever we need to allocate memory from a particular type of allocator, we *push* that allocator onto a global *allocator stack*, and when we're done allocating we must always remember to *pop* the allocator off the stack. To make this more convenient and less error-prone, we use an *allocation janitor*. This tiny class's constructor pushes the allocator, and its destructor pops the allocator:

```
class AllocJanitor
{
public:
    explicit AllocJanitor(mem::Context context)
    {
        mem::PushAllocator(context);
    }
    ~AllocJanitor()
    {
        mem::PopAllocator();
    }
};
```

To use the janitor class, we simply construct a local instance of it. When this instance drops out of scope, the allocator will be popped automatically:

```
void f()
{
    // do some work...

    // allocate temp buffers from single-frame allocator
    {
        AllocJanitor janitor(mem::Context::kSingleFrame);

        U8* pByteBuffer = new U8[SIZE];
        float* pFloatBuffer = new float[SIZE];

        // use buffers...

        // (NOTE: no need to free the memory because we
        // used a single-frame allocator)
    } // janitor pops allocator when it drops out of scope

    // do more work...
}
```

See http://en.cppreference.com/w/cpp/language/raii for more information on the highly useful RAII pattern.

3.1.2 C++ Language Standardization

Since its inception in 1979, the C++ language has been continually evolving. Bjarne Stroustrup, its inventor, originally named the language "C with Classes", but it was renamed "C++" in 1983. The International Organization for Standardization (ISO, www.iso.org) first standardized the language in 1998—this version is known today as C++98. Since then, the ISO has been periodically publishing updated standards for the C++ language, with the goals of making the language more powerful, easier to use, and less ambiguous. These goals are achieved by refining the semantics and rules of the language, by adding new, more-powerful language features, and by deprecating, or removing entirely, those aspects of the language which have proven problematic or unpopular.

The most-recent variant of the C++ programming language standard is called C++17, which was published on July 31, 2017. The next iteration of the standard, C++2a, was in development at the time of this publication. The various versions of the C++ standard are summarized in chronological order below.

- *C++98* was the first official C++ standard, established by the ISO in 1998.
- *C++03* was introduced in 2003, to address various problems that had been identified in the C++98 standard.

- *C++11* (also known as *C++0x* during much of its development) was approved by the ISO on August 12, 2011. C++11 added a large number of powerful new features to the language, including:
  - a type-safe `nullptr` literal, to replace the bug-prone `NULL` macro that had been inherited from the C language;
  - the `auto` and `decltype` keywords for type inference;
  - a "trailing return type" syntax, which allows a `decltype` of a function's input arguments to be used to describe the return type of that function;
  - the `override` and `final` keywords for improved expressiveness when defining and overriding virtual functions;
  - defaulted and deleted functions (allowing the programmer to explicitly request that a compiler-generated default implementation be used, or that a function's implementation should be undefined);
  - delegating constructors—the ability of one constructor to invoke another within the same class;
  - strongly-typed enumerations;
  - the `constexpr` keyword for defining compile-time constant values by evaluating expressions at compile time;
  - a uniform initialization syntax that extends the original braces-based POD initializers to cover non-POD types as well;
  - support for lambda functions and variable capture (closures);
  - the introduction of rvalue references and move semantics for more efficient handling of temporary objects; and
  - standardized attribute specifiers, to replace compiler-specific specifiers such as `__attribute__((...))` and `__declspec()`.

  C++11 also introduced an improved and expanded standard library, including support for threading (concurrent programming), improved smart pointer facilities and an expanded set of generic algorithms.
- *C++14* was approved by the ISO on August 18, 2014 and released on December 15, 2014. Its additions and improvements to C++11 include:
  - return type deduction, which in many situations allows function return types to be declared using a simple `auto` keyword, without the need for the verbose trailing `decltype` expression required by C++11;

  - generic lambdas, allowing a lambda to act like a templated function by using `auto` to declare its input arguments;
  - the ability to initialize "captured" variables in lambdas;
  - binary literals prefaced with `0b` (e.g., `0b10110110`);
  - support for digit separators in numeric literals, for improved readability (e.g., `1'000'000` instead of `1000000`);
  - variable templates, which allow template syntax to be used when declaring variables; and
  - relaxation of some restrictions on `constexpr`, including the ability to use `if`, `switch` and loops within constant expressions.

- *C++17* was published by the ISO on July 31, 2017. It extends and improves C++14 in many ways, including but not limited to the following:
  - removed a number of out-dated and/or dangerous language features, including trigraphs, the `register` keyword, and the already-deprecated `auto_ptr` smart pointer class;
  - guarantees copy *elision*, the omission of unnecessary object copying;
  - exception specifications are now part of the type system, meaning that `void f() noexcept(true);` and `void f() noexcept(false);` are now distinct types;
  - adds two new literals to the language: UTF-8 character literals (e.g., `u8'x'`), and floating-point literals with a hexadecimal base and decimal exponent (e.g., `0xC.68p+2`);
  - introduces *structured bindings* to C++, allowing the values in a collection data type to be "unpacked" into individual variables (e.g., `auto [a, b] = func_that_returns_a_pair();`)—a syntax that is strikingly similar to that of returning multiple values from a function via a tuple in Python;
  - adds some useful standardized attributes, including `[[fallthrough]]` which allows you to explicitly document the fact that a missing `break` statement in a `switch` is intentional, thereby suppressing the warning that would otherwise be generated.

3.1.2.1 Further Reading

There are plenty of great online resources and books that describe the features of C++11, C++14 and C++17 in detail, so we won't attempt to cover them here. Here are a few useful references:

- The site http://en.cppreference.com/w/cpp provides an excellent C++ reference manual, including call-outs such as "since C++11" or "until C++17" to indicate when certain language features were added to or removed from the standard, respectively.
- Information about the ISO's standardization efforts can be found at https://isocpp.org/std.
- See the following sites for good summaries of C++11's major new features:
  - https://www.codeproject.com/Articles/570638/Ten-Cplusplus-Features-Every-Cplusplus-Developer, and
  - https://blog.smartbear.com/development/the-biggest-changes-in-c11-and-why-you-should-care.
- See http://thbecker.net/articles/auto_and_decltype/section_01.html for a great treatment of `auto` and `decltype`.
- See http://www.drdobbs.com/cpp/the-c14-standard-what-you-need-to-know/240169034 for a good treatment of C++14's changes to the C++11 standard.
- See https://isocpp.org/files/papers/p0636r0.html for a complete list of how the C++17 standard differs from C++14.

3.1.2.2 Which Language Features to Use?

As you read about all the cool new features being added to C++, it's tempting to think that you need to use *all* of these features in your engine or game. However, just because a feature exists doesn't mean your team needs to immediately start making use of it.

At Naughty Dog, we tend to take a conservative approach to adopting new language features into our codebase. As part of our studio's coding standards, we have a list of those C++ language features that are approved for use in our runtime code, and another somewhat more liberal list of language features that are allowed in our offline tools code. There are a number of reasons for this cautious approach, which I'll outline in the following sections.

Lack of Full Feature Support

The "bleeding edge" features may not be fully supported by your compiler. For example, LLVM/Clang, the C++ compiler used on the Sony Playstation

4, currently supports the entire C++11 standard in versions 3.3 and later, and all of C++14 in versions 3.4 and later. But its support for C++17 is spread across Clang versions 3.5 through 4.0, and it currently has no support for the draft C++2a standard. Also, Clang compiles code in C++98 mode by default—support for some of the more-advanced standards are accepted as extensions, but in order to enable full support one must pass specific command-line arguments to the compiler. See https://clang.llvm.org/cxx_status.html for details.

Cost of Switching between Standards

There's a non-zero cost to switching your codebase from one standard to another. Because of this, it's important for a game studio to decide on the most-advanced C++ standard to support, and then stick with it for a reasonable length of time (e.g., for the duration of one project). At Naughty Dog, we adopted the C++11 standard only relatively recently, and we only allowed its use in the code branch in which *The Last of Us Part II* is being developed. The code in the branch used for *Uncharted 4: A Thief's End* and *Uncharted: The Lost Legacy* had been written originally using the C++98 standard, and we decided that the relatively minor benefits of adopting C++11 features in that codebase did not outweigh the cost and risks of doing so.

Risk versus Reward

Not every C++ language feature is created equal. Some features are useful and pretty much universally acceptable, like `nullptr`. Others may have benefits, but also associated negatives. Still other language features may be deemed inappropriate for use in runtime engine code altogether.

As an example of a language feature with both benefits and downsides, consider the new C++11 interpretation of the `auto` keyword. This keyword certainly makes variables and functions more convenient to write. But the Naughty Dog programmers recognized that over-use of `auto` can lead to obfuscated code: As an extreme example, imagine trying to read a .cpp file written by somebody else, in which virtually every variable, function argument and return value is declared `auto`. It would be like reading a typeless language such as Python or Lisp. One of the benefits of a strongly-typed language like C++ is the programmer's ability to quickly and easily determine the types of all variables. As such, we decided to adopt a simple rule: `auto` may only be used when declaring iterators, in situations where no other approach works (such as within template definitions), or in special cases when code clarity, readability and maintainability is significantly improved by its use. In all other cases, we require the use of explicit type declarations.

As an example of a language feature that could be deemed inappropriate for use in a commercial product like a game, consider *template metaprogramming*. Andrei Alexandrescu's Loki library [3] makes heavy use of template metaprogramming to do some pretty interesting and amazing things. However, the resulting code is tough to read, is sometimes non-portable, and presents programmers with an extremely high barrier to understanding. The programming leads at Naughty Dog believe that any programmer should be able to jump in and debug a problem on short notice, even in code with which he or she may not be very familiar. As such, Naughty Dog prohibits complex template metaprogramming in runtime engine code, with exceptions made only on a case-by-case basis where the benefits are deemed to outweigh the costs.

In summary, remember that when you have a hammer, everything can tend to look like a nail. Don't be tempted to use features of your language just because they're there (or because they're new). A judicious and carefully considered approach will result in a stable codebase that's as easy as possible to understand, reason about, debug and maintain.

3.1.3 Coding Standards: Why and How Much?

Discussions of coding conventions among engineers can often lead to heated "religious" debates. I do not wish to spark any such debate here, but I will go so far as to suggest that following at least a minimal set of coding standards is a good idea. Coding standards exist for two primary reasons.

1. Some standards make the code more readable, understandable and maintainable.
2. Other conventions help to prevent programmers from shooting themselves in the foot. For example, a coding standard might encourage the programmer to use only a smaller, more testable and less error-prone subset of the whole language. The C++ language is rife with possibilities for abuse, so this kind of coding standard is particularly important when using C++.

In my opinion, the most important things to achieve in your coding conventions are the following.

- *Interfaces are king*. Keep your interfaces (.h files) clean, simple, minimal, easy to understand and well-commented.
- *Good names encourage understanding and avoid confusion*. Stick to intuitive names that map directly to the purpose of the class, function or variable in question. Spend time up-front identifying a good name. Avoid

a naming scheme that requires programmers to use a look-up table in order to decipher the meaning of your code. Remember that high-level programming languages like C++ are intended for *humans* to read. (If you disagree, just ask yourself why you don't write all your software directly in machine language.)

- *Don't clutter the global namespace*. Use C++ namespaces or a common naming prefix to ensure that your symbols don't collide with symbols in other libraries. (But be careful not to overuse namespaces, or nest them too deeply.) Name `#defined` symbols with extra care; remember that C++ preprocessor macros are really just text substitutions, so they cut across all C/C++ scope and namespace boundaries.
- *Follow C++ best practices*. Books like the *Effective C++* series by Scott Meyers [36,37], Meyers' *Effective STL* [38] and *Large-Scale C++ Software Design* by John Lakos [31] provide excellent guidelines that will help keep you out of trouble.
- *Be consistent*. The rule I try to use is as follows: If you're writing a body of code from scratch, feel free to invent any convention you like—then stick to it. When editing preexisting code, try to follow whatever conventions have already been established.
- *Make errors stick out*. Joel Spolsky wrote an excellent article on coding conventions, which can be found at http://www.joelonsoftware.com/articles/Wrong.html. Joel suggests that the "cleanest" code is not necessarily code that looks neat and tidy on a superficial level, but rather the code that is written in a way that makes common programming errors *easier to see*. Joel's articles are always fun and educational, and I highly recommend this one.

3.2 Catching and Handling Errors

There are a number of ways to catch and handle error conditions in a game engine. As a game programmer, it's important to understand these different mechanisms, their pros and cons and when to use each one.

3.2.1 Types of Errors

In any software project there are two basic kinds of error conditions: *user errors* and *programmer errors*. A user error occurs when the user of the program does something incorrect, such as typing an invalid input, attempting to open a file that does not exist, etc. A programmer error is the result of a *bug* in the code itself. Although it may be triggered by something the user has done, the

essence of a programmer error is that the problem could have been avoided if the programmer had not made a mistake, and the user has a reasonable expectation that the program *should* have handled the situation gracefully.

Of course, the definition of "user" changes depending on context. In the context of a game project, user errors can be roughly divided into two categories: errors caused by the person playing the game and errors caused by the people who are making the game during development. It is important to keep track of which type of user is affected by a particular error and handle the error appropriately.

There's actually a third kind of user—the other programmers on your team. (And if you are writing a piece of game middleware software, like Havok or OpenGL, this third category extends to other programmers all over the world who are using your library.) This is where the line between *user errors* and *programmer errors* gets blurry. Let's imagine that programmer A writes a function `f()`, and programmer B tries to call it. If B calls `f()` with invalid arguments (e.g., a null pointer, or an out-of-range array index), then this could be seen as a user error by programmer A, but it would be a programmer error from B's point of view. (Of course, one can also argue that programmer A should have anticipated the passing of invalid arguments and should have handled them gracefully, so the problem really is a programmer error, on A's part.) The key thing to remember here is that the line between user and programmer can shift depending on context—it is rarely a black-and-white distinction.

3.2.2 Handling Errors

When handling errors, the requirements differ significantly between the two types. It is best to handle *user errors* as gracefully as possible, displaying some helpful information to the user and then allowing him or her to continue working—or in the case of a game, to continue playing. Programmer errors, on the other hand, should *not* be handled with a graceful "inform and continue" policy. Instead, it is usually best to halt the program and provide detailed low-level debugging information, so that a programmer can quickly identify and fix the problem. In an ideal world, *all* programmer errors would be caught and fixed before the software ships to the public.

3.2.2.1 Handling Player Errors

When the "user" is the person playing your game, errors should obviously be handled within the context of gameplay. For example, if the player attempts to reload a weapon when no ammo is available, an audio cue and/or an animation can indicate this problem to the player without taking him or her "out of the game."

3.2.2.2 Handling Developer Errors

When the "user" is someone who is making the game, such as an artist, animator or game designer, errors may be caused by an invalid asset of some sort. For example, an animation might be associated with the wrong skeleton, or a texture might be the wrong size, or an audio file might have been sampled at an unsupported sample rate. For these kinds of *developer errors*, there are two competing camps of thought.

On the one hand, it seems important to prevent bad game assets from persisting for too long. A game typically contains literally thousands of assets, and a problem asset might get "lost," in which case one risks the possibility of the bad asset surviving all the way into the final shipping game. If one takes this point of view to an extreme, then the best way to handle bad game assets is to prevent the entire game from running whenever even a single problematic asset is encountered. This is certainly a strong incentive for the developer who created the invalid asset to remove or fix it immediately.

On the other hand, game development is a messy and iterative process, and generating "perfect" assets the first time around is rare indeed. By this line of thought, a game engine should be robust to almost any kind of problem imaginable, so that work can continue even in the face of totally invalid game asset data. But this too is not ideal, because the game engine would become bloated with error-catching and error-handling code that won't be needed once the development pace settles down and the game ships. And the probability of shipping the product with "bad" assets becomes too high.

In my experience, the best approach is to find a middle ground between these two extremes. When a developer error occurs, I like to *make the error obvious* and then allow the team to continue to work in the presence of the problem. It is extremely costly to prevent all the other developers on the team from working, just because one developer tried to add an invalid asset to the game. A game studio pays its employees well, and when multiple team members experience downtime, the costs are multiplied by the number of people who are prevented from working. Of course, we should only handle errors in this way when it is practical to do so, without spending inordinate amounts of engineering time, or bloating the code.

As an example, let's suppose that a particular mesh cannot be loaded. In my view, it's best to draw a big red box in the game world at the places that mesh would have been located, perhaps with a text string hovering over each one that reads, "Mesh *blah-dee-blah* failed to load." This is superior to printing an easy-to-miss message to an error log. And it's *far* better than just crashing the game, because then no one will be able to work until that one mesh

reference has been repaired. Of course, for particularly egregious problems it's fine to just spew an error message and crash. There's no silver bullet for all kinds of problems, and your judgment about what type of error handling approach to apply to a given situation will improve with experience.

3.2.2.3 Handling Programmer Errors

The best way to detect and handle programmer errors (a.k.a. bugs) is often to embed error-checking code into your source code and arrange for failed error checks to halt the program. Such a mechanism is known as an *assertion system*; we'll investigate assertions in detail in Section 3.2.3.3. Of course, as we said above, one programmer's user error is another programmer's bug; hence, assertions are not always the right way to handle every programmer error. Making a judicious choice between an assertion and a more graceful error-handling technique is a skill that one develops over time.

3.2.3 Implementation of Error Detection and Handling

We've looked at some philosophical approaches to handling errors. Now let's turn our attention to the choices we have as programmers when it comes to implementing error detection and handling code.

3.2.3.1 Error Return Codes

A common approach to handling errors is to return some kind of failure code from the function in which the problem is first detected. This could be a Boolean value indicating success or failure, or it could be an "impossible" value, one that is outside the range of normally returned results. For example, a function that returns a positive integer or floating-point value could return a negative value to indicate that an error occurred. Even better than a Boolean or an "impossible" return value, the function could be designed to return an enumerated value to indicate success or failure. This clearly separates the error code from the output(s) of the function, and the exact nature of the problem can be indicated on failure (e.g., `enum Error { kSuccess, kAssetNotFound, kInvalidRange, ... }`).

The calling function should intercept error return codes and act appropriately. It might handle the error immediately. Or, it might work around the problem, complete its own execution and then pass the error code on to whatever function called *it*.

3.2.3.2 Exceptions

Error return codes are a simple and reliable way to communicate and respond to error conditions. However, error return codes have their drawbacks. Perhaps the biggest problem with error return codes is that the function that detects an error may be totally unrelated to the function that is capable of handling the problem. In the worst-case scenario, a function that is 40 calls deep in the call stack might detect a problem that can only be handled by the top-level game loop, or by `main()`. In this scenario, every one of the 40 functions on the call stack would need to be written so that it can pass an appropriate error code all the way back up to the top-level error-handling function.

One way to solve this problem is to throw an exception. *Exception handling* is a very powerful feature of C++. It allows the function that detects a problem to communicate the error to the rest of the code without knowing anything about which function might handle the error. When an exception is thrown, relevant information about the error is placed into a data object of the programmer's choice known as an *exception object*. The call stack is then automatically unwound, in search of a calling function that has wrapped its call in a `try-catch` block. If a `try-catch` block is found, the exception object is matched against all possible `catch` clauses, and if a match is found, the corresponding `catch`'s code block is executed. The destructors of any automatic variables are called as needed during the stack unwinding process.

The ability to separate error detection from error handling in such a clean way is certainly attractive, and exception handling is an excellent choice for some software projects. However, exception handling does add some overhead to the program. The stack frame of any function that contains a `try-catch` block must be augmented to contain additional information required by the stack unwinding process. Also, if even one function in your program (or a library that your program links with) uses exception handling, your entire program must use exception handling—the compiler can't know which functions might be above you on the call stack when you throw an exception.

That said, it is possible to "sandbox" a library or libraries that make use of exception handling, in order to avoid your entire game engine having to be written with exceptions enabled. To do this, you would wrap all the API calls into the libraries in question in functions that are implemented in a translation unit that has exception handling enabled. Each of these functions would catch all possible exceptions in a try/catch block and convert them into error return codes. Any code that links with your wrapper library can therefore safely disable exception handling.

Arguably more important than the overhead issue is the fact that excep-

tions are in some ways no better than `goto` statements. Joel Spolsky of Microsoft and Fog Creek Software fame argues that exceptions are in fact *worse* than `goto`s because they aren't easily seen in the source code. A function that neither throws nor catches exceptions may nevertheless become involved in the stack-unwinding process, if it finds itself sandwiched between such functions in the call stack. And the unwinding process is itself imperfect: Your software can easily be left in an invalid state unless the programmer considers every possible way that an exception can be thrown, and handles it appropriately. This can make writing robust software difficult. When the possibility for exception throwing exists, pretty much every function in your codebase needs to be robust to the carpet being pulled out from under it and all its local objects destroyed whenever it makes a function call.

Another issue with exception handling is its cost. Although in theory modern exception handling frameworks don't introduce additional runtime overhead in the error-free case, this is not necessarily true in practice. For example, the code that the compiler adds to your functions for unwinding the call stack when an exception occurs tends to produce an overall increase in code size. This might degrade I-cache performance, or cause the compiler to decide not to inline a function that it otherwise would have.

Clearly there are some pretty strong arguments for turning *off* exception handling in your game engine altogether. This is the approach employed at Naughty Dog and also on most of the projects I've worked on at Electronic Arts and Midway. In his capacity as Engine Director at Insomniac Games, Mike Acton has clearly stated his objection to the use of exception handling in runtime game code on numerous occasions. JPL and NASA also disallow exception handling in their mission-critical embedded software, presumably for the same reasons we tend to avoid it in the game industry. That said, your mileage certainly may vary. There is no perfect tool and no one right way to do anything. When used judiciously, exceptions *can* make your code easier to write and work with; just be careful out there!

There are many interesting articles on this topic on the web. Here's one good thread that covers most of the key issues on both sides of the debate:

- http://www.joelonsoftware.com/items/2003/10/13.html
- http://www.nedbatchelder.com/text/exceptions-vs-status.html
- http://www.joelonsoftware.com/items/2003/10/15.html

Exceptions and RAII

The "resource acquisition is initialization" pattern (RAII, see Section 3.1.1.6) is often used in conjuction with *exception handling:* The constructor attempts to

acquire the desired resource, and throws an exception if it fails to do so. This is done to avoid the need for an if check to test the status of the object after it has been created—if the constructor returns without throwing an exception, we know for certain that the resource was successfully acquired.

However, the RAII pattern can be used even without exceptions. All it requires is a little discipline to check the status of each new resource object when it is first created. After that, all of the other benefits of RAII can be reaped. (Exceptions can also be replaced by assertion failures to signal the failure of some kinds of resource acquisitions.)

3.2.3.3 Assertions

An *assertion* is a line of code that checks an expression. If the expression evaluates to true, nothing happens. But if the expression evaluates to false, the program is stopped, a message is printed and the debugger is invoked if possible.

Assertions check a programmer's assumptions. They act like *land mines* for bugs. They check the code when it is first written to ensure that it is functioning properly. They also ensure that the original assumptions continue to hold for long periods of time, even when the code around them is constantly changing and evolving. For example, if a programmer changes code that used to work, but accidentally violates its original assumptions, they'll hit the land mine. This immediately informs the programmer of the problem and permits him or her to rectify the situation with minimum fuss. Without assertions, bugs have a tendency to "hide out" and manifest themselves later in ways that are difficult and time-consuming to track down. But with assertions embedded in the code, bugs announce themselves the moment they are introduced—which is usually the best moment to fix the problem, while the code changes that caused the problem are fresh in the programmer's mind. Steve Maguire provides an in-depth discussion of assertions in his must-read book, *Writing Solid Code* [35].

The cost of the assertion checks can usually be tolerated during development, but stripping out the assertions prior to shipping the game can buy back that little bit of crucial performance if necessary. For this reason assertions are generally implemented in such a way as to allow the checks to be stripped out of the executable in non-debug build configurations. In C, an `assert()` macro is provided by the standard library header file `<assert.h>`; in C++, it's provided by the `<cassert>` header.

The standard library's definition of `assert()` causes it to be defined in debug builds (builds with the `DEBUG` preprocessor symbol defined) and

stripped in non-debug builds (builds with the NDEBUG preprocessor symbol defined). In a game engine, you may want finer-grained control over which build configurations retain assertions, and which configurations strip them out. For example, your game might support more than just a debug and development build configuration—you might also have a shipping build with global optimizations enabled, and perhaps even a PGO build for use by profile-guided optimization tools (see Section 2.2.4). Or you might also want to define different "flavors" of assertions—some that are always retained even in the shipping version of your game, and others that are stripped out of the non-shipping build. For these reasons, let's take a look at how you can implement your own ASSERT() macro using the C/C++ preprocessor.

Assertion Implementation

Assertions are usually implemented via a combination of a #defined macro that evaluates to an if/else clause, a function that is called when the assertion fails (the expression evaluates to false), and a bit of assembly code that halts the program and breaks into the debugger when one is attached. Here's a typical implementation:

```
#if ASSERTIONS_ENABLED

// define some inline assembly that causes a break
// into the debugger -- this will be different on each
// target CPU
#define debugBreak() asm { int 3 }

// check the expression and fail if it is false
#define ASSERT(expr) \
    if (expr) { } \
    else \
    { \
        reportAssertionFailure(#expr, \
                 __FILE__, __LINE__); \
        debugBreak(); \
    }

#else

#define ASSERT(expr)  // evaluates to nothing

#endif
```

Let's break down this definition so we can see how it works:

- The outer `#if/#else/#endif` is used to strip assertions from the code base. When `ASSERTIONS_ENABLED` is nonzero, the `ASSERT()` macro is defined in its full glory, and all assertion checks in the code will be included in the program. But when assertions are turned off, `ASSERT(expr)` evaluates to nothing, and all instances of it in the code are effectively removed.
- The `debugBreak()` macro evaluates to whatever assembly-language instructions are required in order to cause the program to halt and the debugger to take charge (if one is connected). This differs from CPU to CPU, but it is usually a single assembly instruction.
- The `ASSERT()` macro itself is defined using a full `if/else` statement (as opposed to a lone `if`). This is done so that the macro can be used in any context, even within *other* unbracketed `if/else` statements.

Here's an example of what would happen if `ASSERT()` were defined using a solitary `if`:

```
// WARNING: NOT A GOOD IDEA!
#define ASSERT(expr)  if (!(expr)) debugBreak()

void f()
{
    if (a < 5)
        ASSERT(a >= 0);
    else
        doSomething(a);
}
```

This expands to the following *incorrect* code:

```
void f()
{
    if (a < 5)
        if (!(a >= 0))
            debugBreak();
        else // oops! bound to the wrong if()!
            doSomething(a);
}
```

- The `else` clause of an `ASSERT()` macro does two things. It displays some kind of message to the programmer indicating what went wrong, and then it breaks into the debugger. Notice the use of `#expr` as the first argument to the message display function. The pound (#) preprocessor operator causes the expression `expr` to be turned into a string, thereby allowing it to be printed out as part of the assertion failure message.

- Notice also the use of `__FILE__` and `__LINE__`. These compiler-defined macros magically contain the .cpp file name and line number of the line of code on which they appear. By passing them into our message display function, we can print the exact location of the problem.

I highly recommend the use of assertions in your code. However, it's important to be aware of their performance cost. You may want to consider defining two kinds of assertion macros. The regular `ASSERT()` macro can be left active in *all* builds, so that errors are easily caught even when not running in debug mode. A second assertion macro, perhaps called `SLOW_ASSERT()`, could be activated only in debug builds. This macro could then be used in places where the cost of assertion checking is too high to permit inclusion in release builds. Obviously `SLOW_ASSERT()` is of lower utility, because it is stripped out of the version of the game that your testers play every day. But at least these assertions become active when programmers are debugging their code.

It's also extremely important to use assertions properly. They should be used to catch bugs in the program itself—*never* to catch user errors. Also, assertions should always cause the entire game to halt when they fail. It's usually a bad idea to allow assertions to be skipped by testers, artists, designers and other non-engineers. (This is a bit like the boy who cried wolf: if assertions can be skipped, then they cease to have any significance, rendering them ineffective.) In other words, assertions should only be used to catch fatal errors. If it's OK to continue past an assertion, then it's probably better to notify the user of the error in some other way, such as with an on-screen message, or some ugly bright-orange 3D graphics.

Compile-Time Assertions

One weakness of assertions, as we've discussed them thus far, is that the conditions encoded within them are only checked at runtime. We have to run the program, *and* the code path in question must actually *execute*, in order for an assertion's condition to be checked.

Sometimes the condition we're checking within an assertion involves information that is entirely known at compile time. For example, let's say we're defining a struct that for some reason needs to be exactly 128 bytes in size. We want to add an assertion so that if another programmer (or a future version of yourself) decides to change the size of the struct, the compiler will give us an error message. In other words, we'd like to write something like this:

```
struct NeedsToBe128Bytes
{
    U32    m_a;
    F32    m_b;
    // etc.
};

// sadly this doesn't work...
ASSERT(sizeof(NeedsToBe128Bytes) == 128);
```

The problem of course is that the `ASSERT()` (or `assert()`) macro needs to be executable at runtime, and one can't even put executable code at global scope in a .cpp file outside of a function definition. The solution to this problem is a *compile-time assertion*, also known as a *static assertion*.

Starting with C++11, the standard library defines a macro named `static_assert()` for us. So we can re-write the example above as follows:

```
struct NeedsToBe128Bytes
{
    U32    m_a;
    F32    m_b;
    // etc.
};

static_assert(sizeof(NeedsToBe128Bytes) == 128,
    "wrong size");
```

If you're not using C++11, you can always roll your own `STATIC_ASSERT()` macro. It can be implemented in a number of different ways, but the basic idea is always the same: The macro places a declaration into your code that (a) is legal at file scope, (b) evaluates the desired expression at compile time rather than runtime, and (c) produces a compile error if and only if the expression is false. Some methods of defining `STATIC_ASSERT()` rely on compiler-specific details, but here's one reasonably portable way to define it:

```
#define _ASSERT_GLUE(a, b)  a ## b
#define ASSERT_GLUE(a, b)   _ASSERT_GLUE(a, b)

#define STATIC_ASSERT(expr) \
    enum \
    { \
        ASSERT_GLUE(g_assert_fail_, __LINE__) \
            = 1 / (int)(!!(expr)) \
    }

STATIC_ASSERT(sizeof(int) == 4);   // should pass
STATIC_ASSERT(sizeof(float) == 1); // should fail
```

This works by defining an anonymous enumeration containing a single enumerator. The name of the enumerator is made unique (within the translation unit) by "gluing" a fixed prefix such as `g_assert_fail_` to a unique suffix—in this case, the line number on which the `STATIC_ASSERT()` macro is invoked. The value of the enumerator is set to `1 / (!!(expr))`. The double negation `!!` ensures that `expr` has a Boolean value. This value is then cast to an `int`, yielding either `1` or `0` depending on whether the expression is `true` or `false`, respectively. If the expression is true, the enumerator will be set to the value `1/1` which is one. But if the expression is false, we'll be asking the compiler to set the enumerator to the value `1/0` which is illegal, and will trigger a compile error.

When our `STATIC_ASSERT()` macro as defined above fails, Visual Studio 2015 produces a compile-time error message like this:

```
1>test.cpp(48): error C2131: expression did not evaluate to
                             a constant
1>  test.cpp(48): note: failure was caused by an undefined
                        arithmetic operation
```

Here's another way to define `STATIC_ASSERT()` using template specialization. In this example, we first check to see if we're using C++11 or beyond. If so, we use the standard library's implementation of `static_assert()` for maximum portability. Otherwise we fall back to our custom implementation.

```
#ifdef __cplusplus
    #if __cplusplus >= 201103L
        #define STATIC_ASSERT(expr) \
            static_assert(expr, \
                          "static assert failed:" \
                          #expr)
    #else
        // declare a template but only define the
        // true case (via specialization)
        template<bool> class TStaticAssert;
        template<> class TStaticAssert<true> {};

        #define STATIC_ASSERT(expr) \
            enum \
            { \
                ASSERT_GLUE(g_assert_fail_, __LINE__) \
                    = sizeof(TStaticAssert<!!(expr)>) \
            }
    #endif
#endif
```

This implementation, using template specialization, may be preferable to the previous one using division by zero, because it produces a slightly better error message in Visual Studio 2015:

```
1>test.cpp(48): error C2027: use of undefined type
                             'TStaticAssert<false>'
1>test.cpp(48): note: see declaration of
                      'TStaticAssert<false>'
```

However, each compiler handles error reporting differently, so your mileage may vary. For more implementation ideas for compile-time assertions, one good reference is http://www.pixelbeat.org/programming/gcc/static_assert.html.

3.3 Data, Code and Memory Layout

3.3.1 Numeric Representations

Numbers are at the heart of everything that we do in game engine development (and software development in general). Every software engineer should understand how numbers are represented and stored by a computer. This section will provide you with the basics you'll need throughout the rest of the book.

3.3.1.1 Numeric Bases

People think most naturally in *base ten*, also known as *decimal notation*. In this notation, ten distinct digits are used (0 through 9), and each digit from right to left represents the next highest power of 10. For example, the number $7803 = (7 \times 10^3) + (8 \times 10^2) + (0 \times 10^1) + (3 \times 10^0) = 7000 + 800 + 0 + 3$.

In computer science, mathematical quantities such as integers and real-valued numbers need to be stored in the computer's memory. And as we know, computers store numbers in *binary* format, meaning that only the two digits 0 and 1 are available. We call this a *base-two* representation, because each digit from right to left represents the next highest power of 2. Computer scientists sometimes use a prefix of "0b" to represent binary numbers. For example, the binary number 0b1101 is equivalent to decimal 13, because $0b1101 = (1 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) = 8 + 4 + 0 + 1 = 13$.

Another common notation popular in computing circles is *hexadecimal*, or *base 16*. In this notation, the 10 digits 0 through 9 and the six letters A through F are used; the letters A through F replace the decimal values 10 through 15, respectively. A prefix of "0x" is used to denote hex numbers in the C and C++